

Homework 11

张子乐, 3230104237

2026-05-13

Example 3.20

1. 背景

目标为最小化函数 $H(x)$, 在 **random walk Metropolis** 算法中, 令

$$X = X^{(t-1)} + \epsilon,$$

其中 ϵ 来自对称分布 (proposal), 则接受概率为

$$\alpha(X | X^{(t-1)}) = \min \{1, \exp [-\beta_t \{H(X) - H(X^{(t-1)})\}]\}.$$

考虑函数

$$H(x) = ((x-1)^2 - 1)^2 + 3s(11.56x^2),$$

其中,

$$s(x) = \begin{cases} |x| \bmod 2 & \text{for } 2k \leq |x| \leq 2k+1 \\ 2 - |x| \bmod 2 & \text{for } 2k+1 \leq |x| \leq 2(k+1) \end{cases}$$

k 是自然数。

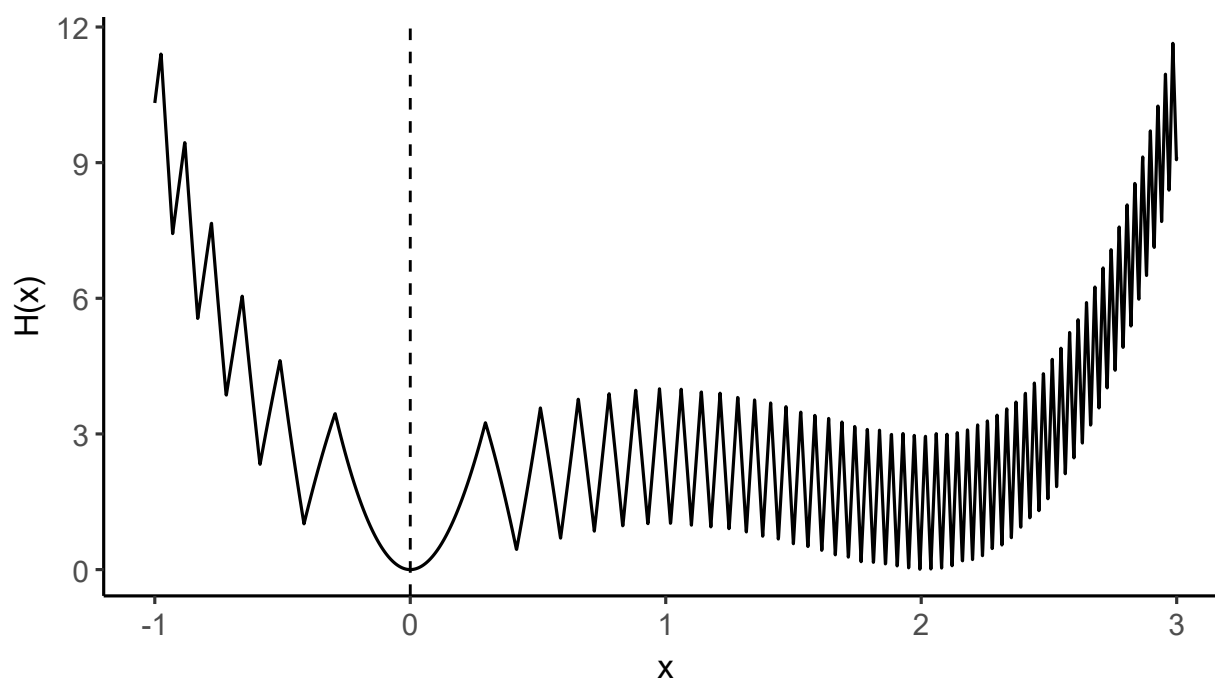
在 $[-1, 3]$ 上, $H(x)$ 的全局最小值点为 $x = 0$ 。下面分别使用 Cauchy proposal 和 Gaussian proposal 进行 simulated annealing。

```
s_fun <- function(z) {  
  r <- abs(z) %% 2  
  ifelse(r <= 1, r, 2 - r)  
} # s(x)  
  
H_fun <- function(x) {  
  ((x - 1)^2 - 1)^2 + 3 * s_fun(11.56 * x^2)  
} # H(x)
```

```

# 画出目标函数  $H(x)$  在  $[-1, 3]$  上的图像
x_grid <- seq(-1, 3, length.out = 4000)
h_grid <- H_fun(x_grid)
df_h <- data.frame(x = x_grid, H = h_grid)
ggplot(df_h, aes(x = x, y = H)) +
  geom_line(linewidth = 0.55, color = "black") +
  geom_vline(xintercept = 0, linetype = "dashed", color = "black") +
  labs(
    x = "x",
    y = "H(x)"
  ) +
  coord_cartesian(xlim = c(-1, 3)) +
  theme_classic(base_size = 12) +
  theme(
    axis.title = element_text(size = 12),
    axis.text = element_text(size = 11)
  )

```



2. 算法设计

采用 geometric tempering:

$$\beta_0 = 1, \quad \beta_t = 1.001\beta_{t-1}.$$

算法从 $x_0 = 1.8$ 开始, 迭代 3000 次。为了比较 proposal 分布的影响, 分别令

$$\epsilon \sim \text{Cauchy}(0, \sqrt{0.1})$$

和

$$\epsilon \sim N(0, 0.01)$$

```
# 模拟退火算法实现
run_sa <- function(n_iter = 3000, x0 = 1.8, beta0 = 1,
                  beta_growth = 1.001, proposal = c("cauchy", "gaussian"),
                  scale = sqrt(0.1),
                  beta_update = function(t, beta0, beta_growth, beta_prev) {
                    beta0 * beta_growth^t
                  }) {
  proposal <- match.arg(proposal)
  if (!is.function(beta_update)) {
    stop("beta_update must be a function.")
  }

  x <- numeric(n_iter + 1)
  h <- numeric(n_iter + 1)
  beta <- numeric(n_iter + 1)
  accepted <- logical(n_iter + 1)
  x[1] <- x0 # 初始点
  h[1] <- H_fun(x0) # 初始点的目标函数值
  beta[1] <- beta0 # 初始 beta(inverse temperature)

  for (t in 1:n_iter) {
    beta_now <- beta_update(
      t = t,
      beta0 = beta0,
      beta_growth = beta_growth,
      beta_prev = beta[t]
    ) # 增加 beta
    if (!is.numeric(beta_now) || length(beta_now) != 1 || !is.finite(beta_now)) {
      stop("beta_update must return one finite numeric value.")
    }

    eps <- switch(
      proposal,
      cauchy = rcauchy(1, location = 0, scale = scale),
```

```

    gaussian = rnorm(1, mean = 0, sd = scale)
  )
  x_candidate <- x[t] + eps # 新的候选值
  h_candidate <- H_fun(x_candidate)
  log_alpha <- -beta_now * (h_candidate - h[t]) # 计算接受概率的对数

  if (log(runif(1)) < min(0, log_alpha)) {
    x[t + 1] <- x_candidate
    h[t + 1] <- h_candidate
    accepted[t + 1] <- TRUE
  } else {
    x[t + 1] <- x[t]
    h[t + 1] <- h[t]
  }
  beta[t + 1] <- beta_now
}

data.frame(
  t = 0:n_iter,
  x = x,
  H = h,
  beta = beta,
  accepted = accepted,
  proposal = proposal
)
}

```

```

# 画图函数：展示  $X[t]$  和  $H(X[t])$  随时间  $t$  的变化
trace_plot <- function(sa_result, plot_title = NULL) {
  old_par <- par(no.readonly = TRUE)
  on.exit(par(old_par))

  xlim <- range(sa_result$t)
  ylim_x <- extendrange(c(sa_result$x, 0), f = 0.04)
  ylim_h <- extendrange(c(sa_result$H, 0), f = 0.04)

  par(
    mfrow = c(2, 1),
    mar = c(0.8, 5, 1, 1),

```

```

    oma = c(4, 0, 0, 0),
    mgp = c(3, 0.8, 0),
    tcl = -0.4
)

plot(
  sa_result$t, sa_result$x,
  type = "s",
  xlim = xlim,
  ylim = ylim_x,
  xaxt = "n",
  xlab = "",
  ylab = expression(X((t))),
  col = "black",
  lwd = 1
)
abline(h = 0, lty = 2, col = "black")

par(mar = c(3, 5, 0.8, 1))
plot(
  sa_result$t, sa_result$H,
  type = "s",
  xlim = xlim,
  ylim = ylim_h,
  xlab = "",
  ylab = expression(H(X((t)))),
  col = "black",
  lwd = 1
)
abline(h = 0, lty = 2, col = "black")
mtext(expression(t), side = 1, outer = TRUE, line = 2.2)

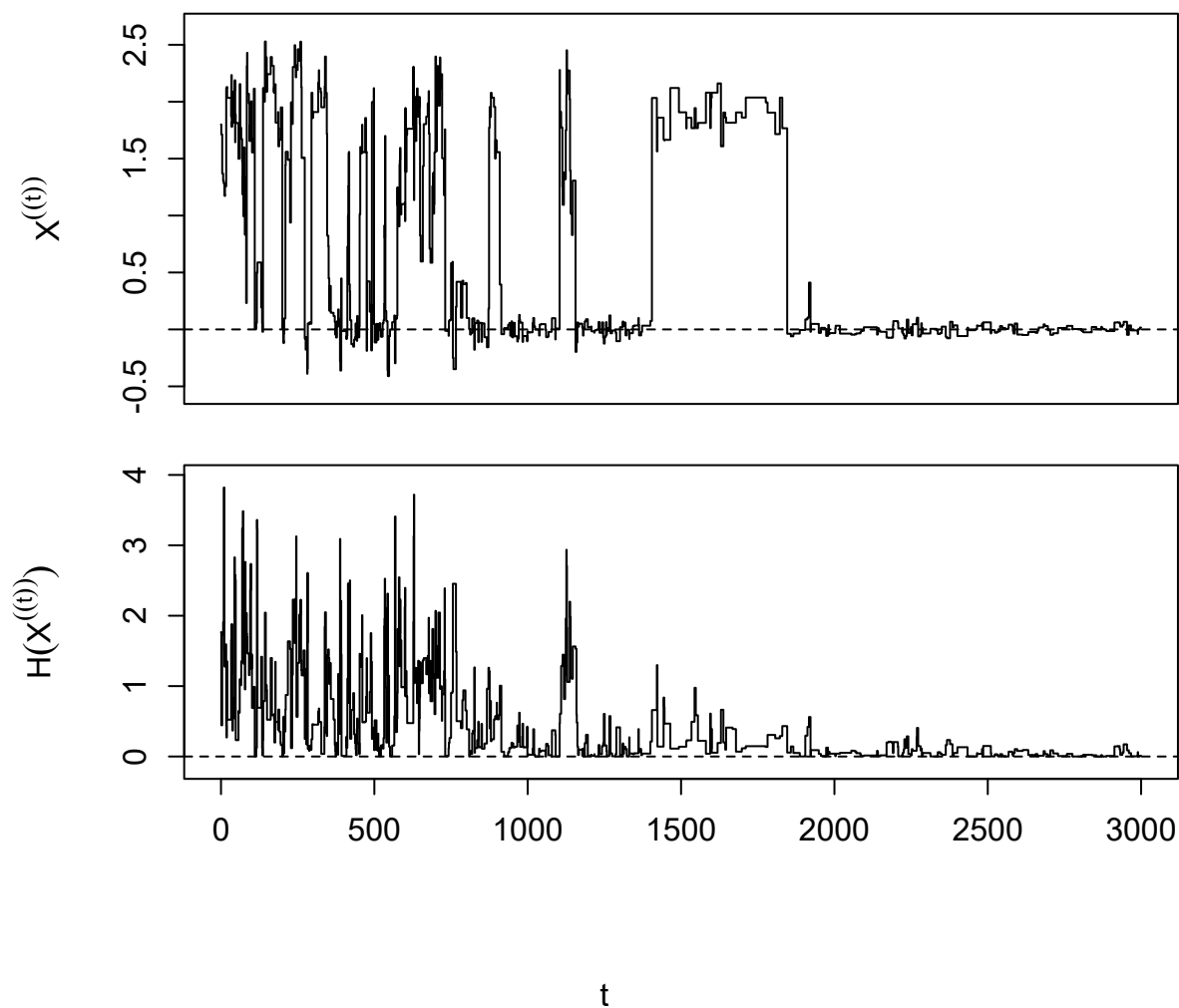
invisible(plot_title)
}

```

3. Cauchy proposal

```
set.seed(42)
sa_cauchy <- run_sa(proposal = "cauchy")
cat("Cauchy proposal 的接受率:", mean(sa_cauchy$accepted[-1]), "\n")
Cauchy proposal 的接受率: 0.192
cat(" 最后一次迭代的  $X_t$ :", tail(sa_cauchy$x, 1), "\n")
最后一次迭代的  $X_t$ : 0.0162894
cat(" 最后一次迭代的  $H(X_t)$ :", tail(sa_cauchy$H, 1), "\n")
最后一次迭代的  $H(X_t)$ : 0.01024631
```

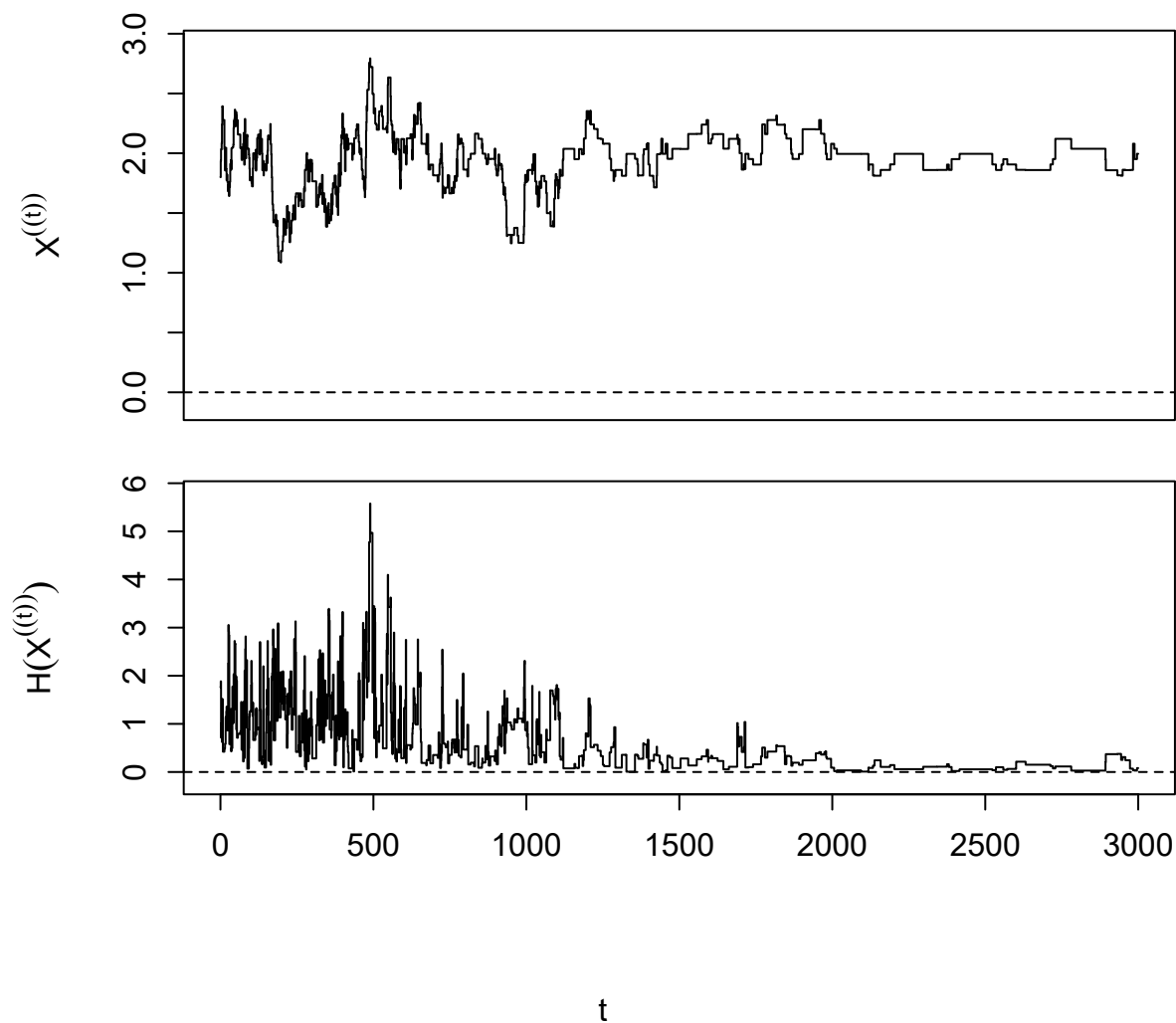
```
trace_plot(sa_cauchy, "Cauchy proposal")
```



4. Gaussian proposal

```
set.seed(42)
sa_gaussian <- run_sa(proposal = "gaussian", scale = sqrt(0.01)) # 取较小的方差
cat("Gaussian proposal 的接受率:", mean(sa_gaussian$accepted[-1]), "\n")
Gaussian proposal 的接受率: 0.2
cat(" 最后一次迭代的  $X_t$ :", tail(sa_gaussian$x, 1), "\n")
最后一次迭代的  $X_t$ : 1.995385
cat(" 最后一次迭代的  $H(X_t)$ :", tail(sa_gaussian$H, 1), "\n")
最后一次迭代的  $H(X_t)$ : 0.08063768
```

```
trace_plot(sa_gaussian, "Gaussian proposal")
```



5. 小结

Cauchy proposal 在迭代次数为 1500 到 2000 之间曾一度在局部最小值点 $x = 1.8$ 附近徘徊，但最终成功跳出并在 2000 步后逐渐接近全局最小值点 $x = 0$ 附近。

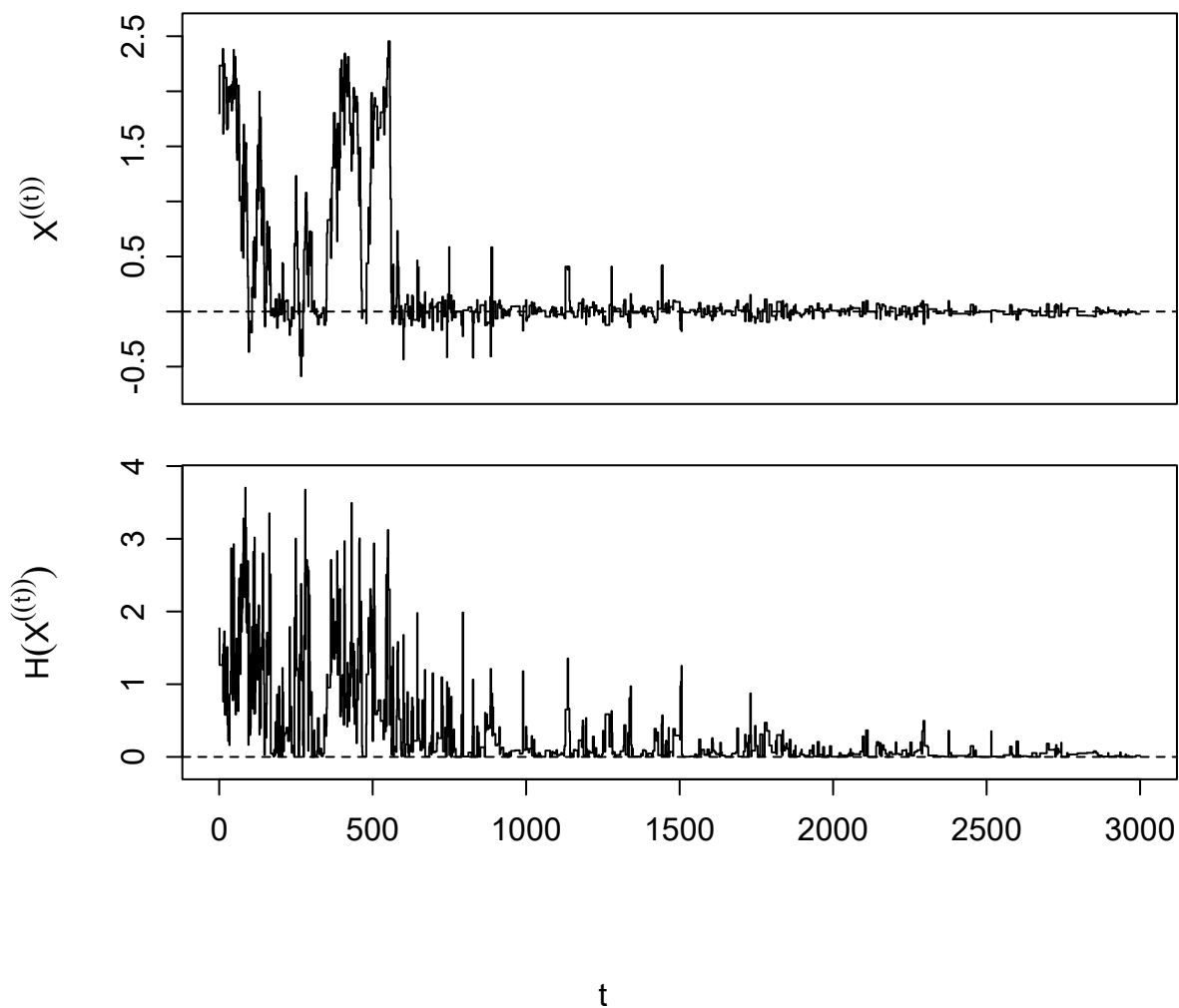
Gaussian proposal 则在迭代过程中一直停留在 $x = 1.8$ 附近，未能成功跳出局部最优。

正如教材中所述：

Note that when using a Gaussian distribution with small enough a variance the simulated annealing algorithm is very likely to remain in the local minimum at $x \approx 1.8$.

Cauchy proposal 的尾部较厚，因此即使在较低温度下，也仍然可能产生较大的跳跃，从而跳出局部最优。Gaussian proposal 方差较小时，大部分 proposal 都集中在当前点附近，因此更容易陷入局部最优。事实上当取一个较大的方差 $\epsilon \sim N(0, 0.1)$ 时，Gaussian proposal 也能成功跳出局部最优，最终接近全局最小值点 $x = 0$ 附近。

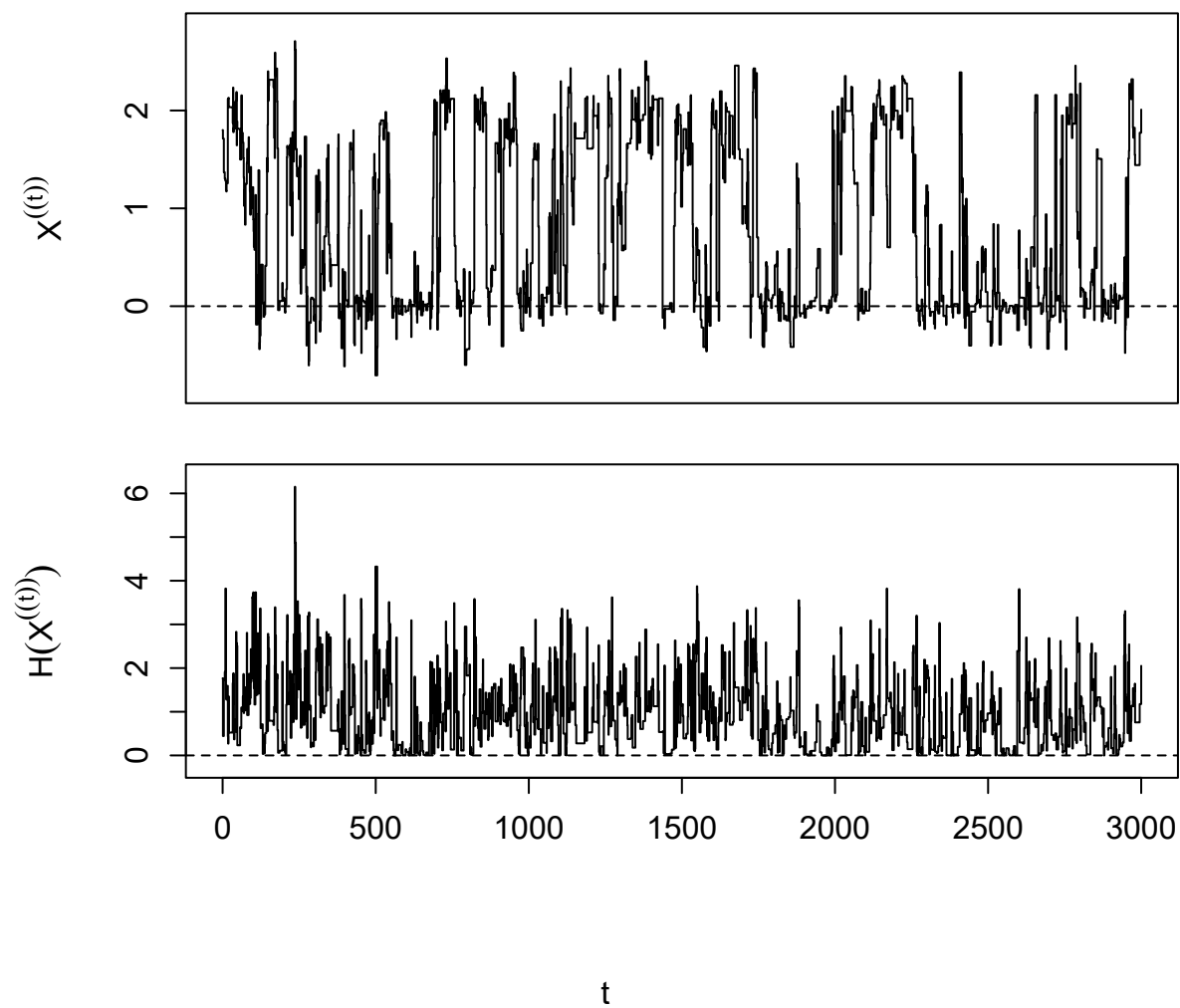
```
set.seed(42)
sa_gaussian2 <- run_sa(proposal = "gaussian", scale = sqrt(0.1)) # 取较大的方差
trace_plot(sa_gaussian2, "Gaussian proposal")
```



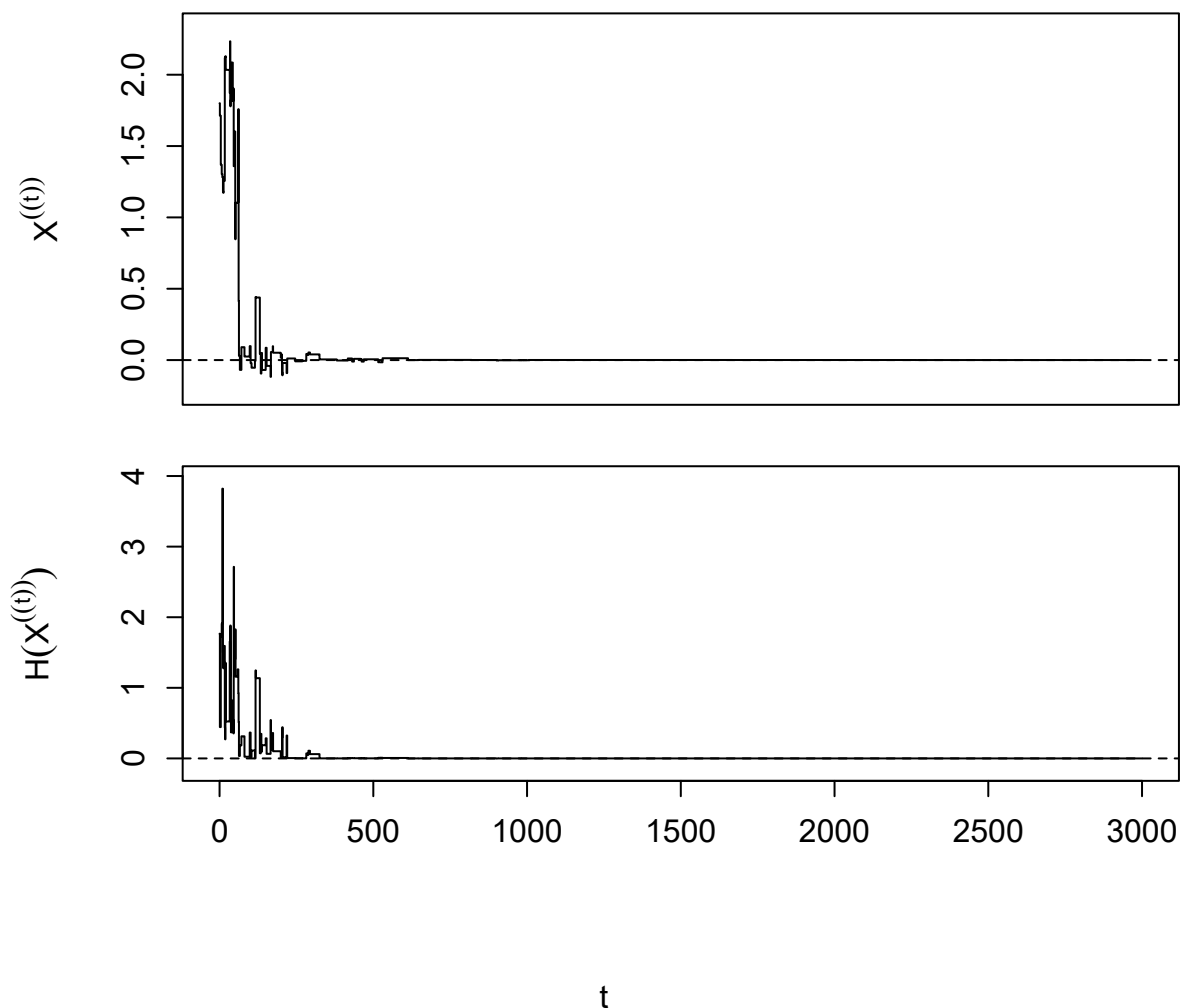
6. 比较不同 beta 的更新方式/速度

在 cauchy proposal 上进行比较。首先仍然使用 geometric tempering: $\beta_0 = 1$, $\beta_t = \alpha\beta_{t-1}$. 之前取 $\alpha = 1.001$, 现在分别取 $\alpha = 1.0001$ 和 $\alpha = 1.01$ 进行比较。

```
set.seed(42)
cauchy2 <- run_sa(proposal = "cauchy", beta_growth = 1.0001) # 取较小的 alpha
trace_plot(cauchy2, "Cauchy proposal")
```



```
set.seed(42)
cauchy3 <- run_sa(proposal = "cauchy", beta_growth = 1.01) # 取较大的 alpha
trace_plot(cauchy3, "Cauchy proposal")
```



可以看到 $\alpha = 1.0001$ 的情况下，beta 增长较慢，算法更容易接受不如当前点的解，经过 3000 次迭代后仍然剧烈震荡。

而 $\alpha = 1.01$ 的情况下，beta 增长较快，算法以更快的速度接近全局最小值点 $x = 0$ 。理论上如果初期 β 过大，即温度过低，算法可能过早地陷入局部最优。但在该随机数种子下尝试 $\alpha = 3, 5, 10$ 时，算法仍然成功地跳出了局部最优。由于篇幅原因，这里不再展示对应的图像。

下面尝试 logarithmic tempering: $\beta_t = \log(1 + t)/\beta_0$ 。讲义中提到，

In other words, the reason why simulated annealing with logarithmic tempering works, is that it still behaves very much like an exhaustive search. However the only reason why we consider simulated annealing is that exhaustive search would be too slow!

事实上从图像上来看，logarithmic tempering 的情况下，算法稳定的速度并不慢于取较小 α 的 geometric tempering。

```

# logarithmic tempering
cauchy4 <- run_sa(
  proposal = "cauchy",
  beta_update = function(t, beta0, ...) log(1 + t) / beta0
)
trace_plot(cauchy4, "Cauchy proposal")

```

